

МИНИСТЕРСТВО ЛЕСНОГО ХОЗЯЙСТВА И ОХРАНЫ ОБЪЕКТОВ ЖИВОТНОГО МИРА НИЖЕГОРОДСКОЙ
ОБЛАСТИ

Государственное бюджетное профессиональное образовательное
учреждение Нижегородской области

«КРАСНОБАКОВСКИЙ ЛЕСНОЙ КОЛЛЕДЖ»

(ГБПОУ НО «КБЛК»)

День 12.

**«Отладка программы с использованием специализированных средств
отладки».**

Цель: изучить способы отладки программы с использованием языка
программирования Python.

Выполнил: _____,

Студент 24-ИС.

2026г.

Выявление ошибок.

Таблица 1. Варианты задания и нужные ошибки.

Вариант	Структура данных	Ошибка индексации	Логическая ошибка	Тип утечки памяти	Особенность
5	Генераторы	StopIteration без обработки	Неверный сдвиг бит (>> вместо <<)	Замыкание с большими данными	Бесконечный генератор

Подробное описание ошибок

1. StopIteration без обработки - генератор выдает 3 элемента, программа запрашивает 5, возникает исключение, которое перехватывается но не обрабатывается корректно, программа продолжает работать с мусорными данными.
2. Неверный сдвиг бит - перепутан оператор сдвига: используется >> (деление на 2) вместо << (умножение на 2), что дает неправильные математические результаты.
3. Утечка памяти через замыкание - внутренняя функция захватывает большой список (1 млн элементов) через замыкание, которое сохраняется в глобальном кеше, не позволяя сборщику мусора освободить память после использования.
4. Бесконечный генератор- цикл while True без условия выхода, генератор никогда не завершается, программа зависает.

Исправление ошибок в коде

1. StopIteration без обработки

Вместо того чтобы просто перехватывать исключение и продолжать работу с мусорными данными, добавил корректную обработку с выходом из цикла при возникновении StopIteration. Теперь программа понимает, что генератор исчерпан, и завершает итерацию, не подставляя некорректные значения.

2. Неверный сдвиг бит

Заменяю оператор сдвига вправо >> на оператор сдвига влево <<, что обеспечивает правильное умножение чисел на 2. Также добавил условие выхода из генератора, чтобы он не работал бесконечно.

3. Утечка памяти через замыкание

Вместо глобального списка, который бесконтрольно накапливал замыкания с большими данными, создал класс MemoryEfficientCache с ограничением размера (max_size=10). Теперь при превышении лимита старые элементы автоматически удаляются. Также добавил явное удаление больших объектов через del после извлечения нужных данных.

4. Бесконечный генератор

Добавил параметр `max_items` и условие выхода `while num < max_items` бесконечного `while True`. Теперь генератор завершается после генерации заданного количества элементов, что предотвращает зависание программы.

Проверка исправленной программы через cmd.

1. Запуск программы

```
python after_var_5.py
```

2. Запуск под отладчиком pdb

```
python -m pdb after_var_5.py
```

```
python -m pdb -c continue after_var_5.py
```

3. Проверка использования памяти с tracemalloc

```
python -c "import tracemalloc; exec(open(after_var_5.py).read())"
```

4. Профилирование с cProfile

```
python -m cProfile after_var_5.py
```

```
python -m cProfile -s cumtime after_var_5.py
```

5. Профилирование памяти с memory_profiler

```
python -m memory_profiler after_var_5.py
```

```
pip install memory_profiler
```

```
python -m memory_profiler after_var_5.py
```

6. Проверка синтаксиса

```
python -m py_compile after_var_5.py
```

```
python -c "import ast; ast.parse(open(after_var_5.py).read())"
```

7. Статистика памяти через tracemalloc с топ-10 строками

```
python -c "import tracemalloc; tracemalloc.start(); exec(open(after_var_5.py).read()); snapshot = tracemalloc.take_snapshot(); [print(stat) for stat in snapshot.statistics(lineno)[:10]]"
```

8. Запуск с увеличенной глубиной рекурсии

```
python -c "import sys; sys.setrecursionlimit(5000); exec(open(after_var_5.py).read())"
```

9. Запуск с отключенным breakpoint()

```
set PYTHONBREAKPOINT=0 && python after_var_5.py
```

```
$env:PYTHONBREAKPOINT=0; python after_var_5.py
```

10. Проверка утечек памяти с трассировкой

```
python -c "import tracemalloc; tracemalloc.start(); exec(open(after_var_5.py).read()); snapshot = tracemalloc.take_snapshot(); print(Топ-5:, snapshot.statistics(filename)[:5])"
```

11. Запуск с логированием

```
python -c "import logging; logging.basicConfig(level=logging.DEBUG);
exec(open(after_var_5.py).read())"
```

12. Комплексная проверка

```
python -c "import tracemalloc, time; tracemalloc.start(); start=time.time();
exec(open(after_var_5.py).read()); print(fВремя: {time.time()-start}с);
snapshot=tracemalloc.take_snapshot(); print(Память:, sum(stat.size for stat in
snapshot.statistics(lineno)[:5])/1024/1024, МБ)"
```

Код с ошибками.

```
import tracemalloc
import time
import sys
from typing import Generator, List, Dict, Any

# Глобальный кеш для хранения больших данных (утечка памяти)
BIG_DATA_CACHE = []

def create_large_data() -> List[int]:
    """Создает большой список данных (замыкание)"""
    # Потенциальная утечка: большие данные сохраняются в замыкании
    large_list = [i * 2 for i in range(1000000)]

    def inner_function() -> List[int]:
        # Замыкание захватывает large_list, препятствуя сборке мусора
        return large_list[:100] # Возвращаем только часть, но весь список в
        памяти

    # Утечка: сохраняем замыкание в глобальном кеше
    BIG_DATA_CACHE.append(inner_function)
    return large_list

def bit_generator_wrong() -> Generator[int, None, None]:
    """
    Генератор с логической ошибкой: неверный сдвиг бит
    Ошибка: используется >> (сдвиг вправо) вместо << (сдвиг влево)
    """
    num = 1
    while True:
        # Ошибка: должен быть num << 1, а не num >> 1
        yield num >> 1 # Неверный сдвиг!
        num += 1

    # Условный выход, чтобы не был бесконечным
    if num > 10:
        break
```

```

def infinite_generator() -> Generator[int, None, None]:
    """
    Бесконечный генератор без условия выхода
    Ошибка: отсутствует условие остановки
    """
    num = 0
    while True:
        # Нет условия выхода - бесконечный цикл
        yield num ** 2
        num += 1

def process_data_with_generators(data: List[Dict[str, Any]]) -> Dict[str, Any]:
    """
    Основная функция обработки данных с использованием генераторов
    """
    result = {
        even_squares: [],
        odd_squares: [],
        bit_results: [],
        processing_time: 0
    }

    start_time = time.time()

    # Ошибка 1: StopIteration без обработки
    # Создаем генератор, который закончится быстрее, чем мы ожидаем
    def limited_generator():
        for i in range(3): # Всего 3 элемента
            yield i * 10

    gen = limited_generator()

    # Пытаемся получить больше элементов, чем есть в генераторе
    # Это вызовет StopIteration
    try:
        for i in range(5): # Запрашиваем 5 элементов, а есть только 3
            value = next(gen)
            result[bit_results].append(value)
            print(f"Получено значение из генератора: {value}")
    except StopIteration as e:
        # Ошибка: мы не обрабатываем StopIteration корректно
        # Просто выводим сообщение, но не обрабатываем ситуацию
        print(f"Ошибка StopIteration: генератор закончился раньше времени")
        # Утечка: продолжаем выполнение с некорректными данными
        result[bit_results].append(-1) # Заполняем мусорными данными

    # Ошибка 2: Неверный сдвиг бит
    bit_gen = bit_generator_wrong()

```

```

bit_values = []
try:
    for _ in range(10):
        bit_values.append(next(bit_gen))
except StopIteration:
    pass

# Сравнение правильного и неправильного результата
print("\n=== Ошибка в битовом сдвиге ===")
print(f"Неправильные значения (>>): {bit_values[:10]}")
correct_values = [1 << 1, 2 << 1, 3 << 1, 4 << 1, 5 << 1, 6 << 1, 7 << 1, 8
<< 1, 9 << 1, 10 << 1]
print(f"Правильные значения (<<): {correct_values}")

# Ошибка 3: Утечка памяти через замыкание
print("\n=== Утечка памяти через замыкание ===")
large_data = create_large_data()
print(f"Создан большой список размером: {sys.getsizeof(large_data)} байт")
print(f"Количество замыканий в кеше: {len(BIG_DATA_CACHE)}")

# Ошибка 4: Бесконечный генератор
print("\n=== Бесконечный генератор ===")
inf_gen = infinite_generator()

# Попытка получить значения из бесконечного генератора
# Ограничиваем количество итераций, чтобы не зависнуть
inf_values = []
try:
    for i in range(10):
        inf_values.append(next(inf_gen))
        if i == 9:
            # Имитация ошибки: забыли добавить условие выхода
            print("Бесконечный генератор продолжает работать...")
            # В реальном коде здесь бы был бесконечный цикл
            break
except Exception as e:
    print(f"Ошибка в бесконечном генераторе: {e}")

result[even_squares] = [x for x in inf_values if x % 2 == 0]
result[odd_squares] = [x for x in inf_values if x % 2 != 0]
result[bit_results].extend(bit_values[:5])

result[processing_time] = time.time() - start_time

return result

def test_generator_errors():
    """
    Тестовая функция для демонстрации всех ошибок
    """

```

```

print("=" * 60)
print("ЗАПУСК ТЕСТА ГЕНЕРАТОРОВ (Вариант №5)")
print("=" * 60)

# Запуск отслеживания памяти
tracemalloc.start()

# Тестовые данные
test_data = [
    {id: 1, value: 100},
    {id: 2, value: 200},
    {id: 3, value: 300},
    # Добавляем данные, которые могут вызвать проблемы
    {id: 4, value: 400},
    {id: 5, value: 500}
]

try:
    # Вызов основной функции
    result = process_data_with_generators(test_data)

    print("\n=== РЕЗУЛЬТАТ ОБРАБОТКИ ===")
    print(f"Четные квадраты: {result[even_squares]}")
    print(f"Нечетные квадраты: {result[odd_squares]}")
    print(f"Результаты битового сдвига: {result[bit_results]}")
    print(f"Время обработки: {result[processing_time]:.4f} сек")

    # Демонстрация StopIteration
    print("\n=== ДЕМОНСТРАЦИЯ STOPITERATION ===")
    gen = (x for x in range(2))
    print(f"Элемент 1: {next(gen)}")
    print(f"Элемент 2: {next(gen)}")
    # Следующий вызов вызовет StopIteration
    try:
        print(f"Элемент 3: {next(gen)}")
    except StopIteration:
        print("Перехвачено StopIteration (как и ожидалось)")

except Exception as e:
    print(f"\n!!! КРИТИЧЕСКАЯ ОШИБКА: {e}")
    import traceback
    traceback.print_exc()

finally:
    # Снимок памяти
    print("\n=== СТАТИСТИКА ПАМЯТИ ===")
    snapshot = tracemalloc.take_snapshot()

    print("\nТоп-10 строк по потреблению памяти:")
    for i, stat in enumerate(snapshot.statistics(lineno)[:10], 1):
        print(f"{i}. {stat}")

```

```

# Информация о размере данных в кеше
total_cache_size = sum(sys.getsizeof(cache) for cache in BIG_DATA_CACHE)
print(f"\nРазмер глобального кеша: {total_cache_size} байт")
print(f"Количество объектов в кеше: {len(BIG_DATA_CACHE)}")

tracemalloc.stop()

def debug_with_pdb():
    """
    Функция для демонстрации отладки с pdb
    """
    print("\n=== ДЕМОНСТРАЦИЯ ОТЛАДКИ С PDB ===")
    print("Для использования pdb раскомментируйте строку с breakpoint()")

    # Раскомментируйте для отладки:
    # breakpoint() # Точка останова

    # Пример проблемного генератора
    def problematic_generator():
        data = [1, 2, 3, 4, 5]
        for item in data:
            # Здесь происходит ошибка: выход за границы
            try:
                # Попытка получить следующий элемент (вызовет StopIteration)
                next_item = data[data.index(item) + 1]
                yield item * next_item
            except IndexError:
                # Ошибка: неправильная обработка IndexError
                # Вместо корректного завершения, просто игнорируем
                print(f"Ошибка индекса для элемента {item}")
                yield -1

    gen = problematic_generator()
    print("Результаты проблемного генератора:")
    for _ in range(10): # Запрашиваем больше элементов, чем есть
        try:
            value = next(gen)
            print(f" {value}")
        except StopIteration:
            print(" Генератор завершен")
            break

if __name__ == "__main__":
    # Запуск основной тестовой функции
    test_generator_errors()

    # Демонстрация отладки
    debug_with_pdb()

```

```

print("\n" + "=" * 60)
print("АНАЛИЗ ОШИБОК В ВАРИАНТЕ №5")
print("=" * 60)
print("""
1. StopIteration без обработки
2. Неверный сдвиг бит
3. Утечка памяти через замыкание
4. Бесконечный генератор
""")

```

Исправленный код

```

import tracemalloc
import time
import sys
from typing import Generator, List, Dict, Any
from functools import lru_cache
import weakref

class MemoryEfficientCache:
    """
    Класс для управления кешем с автоматической очисткой
    Вместо глобального списка используем LRU кеш с ограничением
    """
    def __init__(self, max_size: int = 100):
        self.max_size = max_size
        self._cache = {}
        self._access_order = []

    def add(self, key: Any, value: Any) -> None:
        """Добавление значения в кеш с контролем размера"""
        if len(self._cache) >= self.max_size:
            # Удаляем самый старый элемент
            oldest_key = self._access_order.pop(0)
            del self._cache[oldest_key]

        self._cache[key] = value
        if key in self._access_order:
            self._access_order.remove(key)
        self._access_order.append(key)

    def get(self, key: Any, default: Any = None) -> Any:
        """Получение значения из кеша"""
        if key in self._cache:
            # Обновляем порядок доступа
            self._access_order.remove(key)
            self._access_order.append(key)
            return self._cache[key]

```

```

        return default

    def clear(self) -> None:
        """Очистка кеша"""
        self._cache.clear()
        self._access_order.clear()

    def size(self) -> int:
        """Текущий размер кеша"""
        return len(self._cache)

# Используем класс вместо глобального списка
BIG_DATA_CACHE = MemoryEfficientCache(max_size=10)

def create_large_data_efficient() -> List[int]:
    """
    Создание больших данных без утечки памяти
    Используем weakref или немедленное освобождение
    """
    # Создаем большой список
    large_list = [i * 2 for i in range(1000000)]

    # Извлекаем нужные данные
    result = large_list[:100]

    # Явно удаляем большой список для освобождения памяти
    del large_list

    # Сохраняем только результат в кеше (уже не большой список)
    BIG_DATA_CACHE.add(last_result, result)

    return result

def create_large_data_with_weakref() -> List[int]:
    """
    Альтернативный подход с использованием слабых ссылок
    """
    large_list = [i * 2 for i in range(1000000)]

    # Используем слабую ссылку вместо сильной
    weak_list = weakref.ref(large_list)

    # Возвращаем только необходимые данные
    result = large_list[:100]

    # Удаляем оригинальный список, слабая ссылка не мешает сборке мусора
    del large_list

```

```

    return result

def fixed_bit_generator() -> Generator[int, None, None]:
    """
    Исправленный генератор с правильным сдвигом бит
    """
    num = 1
    while num <= 10: # Добавлено ограничение
        yield num << 1 # Правильный сдвиг влево (умножение на 2)
        num += 1

def safe_generator_with_validation(data: List[int]) -> Generator[int, None,
None]:
    """
    Генератор с валидацией и безопасной обработкой
    """
    for item in data:
        if item is None:
            continue # Пропускаем None значения
        try:
            # Валидация данных перед обработкой
            if not isinstance(item, (int, float)):
                raise TypeError(f"Неверный тип данных: {type(item)}")
            yield item * 2
        except (TypeError, ValueError) as e:
            print(f"Ошибка при обработке {item}: {e}")
            # Возвращаем значение по умолчанию вместо ошибки
            yield 0

def limited_infinite_generator(max_items: int = 100) -> Generator[int, None,
None]:
    """
    Исправленный генератор с условием выхода
    """
    num = 0
    while num < max_items: # Добавлено условие выхода
        yield num ** 2
        num += 1

def process_data_with_generators_fixed(data: List[Dict[str, Any]]) -> Dict[str,
Any]:
    """
    Исправленная основная функция обработки данных
    """
    result = {
        even_squares: [],
        odd_squares: [],
    }

```

```

        bit_results: [],
        processing_time: 0,
        cache_size: 0
    }

    start_time = time.time()

    # Используем безопасный генератор с обработкой StopIteration
    def safe_generator():
        for i in range(5): # Увеличили количество элементов
            yield i * 10

    gen = safe_generator()

    # Корректная обработка StopIteration
    for i in range(7): # Запрашиваем 7 элементов
        try:
            value = next(gen)
            result[bit_results].append(value)
            print(f"Получено значение из генератора: {value}")
        except StopIteration:
            print(f"Генератор закончился на итерации {i}. Больше элементов нет.")
            break # Выходим из цикла корректно

    # Используем исправленный генератор битового сдвига
    bit_gen = fixed_bit_generator()
    bit_values = list(bit_gen) # Безопасное получение всех значений

    print("\n=== ИСПРАВЛЕННЫЙ БИТОВЫЙ СДВИГ ===")
    print(f"Правильные значения (<<): {bit_values}")

    # Исправленная утечка памяти
    print("\n=== ИСПРАВЛЕННАЯ УТЕЧКА ПАМЯТИ ===")
    large_data = create_large_data_efficient()
    print(f"Создан список размером: {sys.getsizeof(large_data)} байт")
    print(f"Количество элементов в кеше: {BIG_DATA_CACHE.size()}")

    # Исправленный бесконечный генератор
    print("\n=== ИСПРАВЛЕННЫЙ ГЕНЕРАТОР ===")
    inf_gen = limited_infinite_generator(10) # Ограничили 10 элементами

    # Безопасное получение значений
    inf_values = []
    for i, value in enumerate(inf_gen):
        inf_values.append(value)
        print(f"Квадрат {i}: {value}")

    # Обработка результатов
    result[even_squares] = [x for x in inf_values if x % 2 == 0]
    result[odd_squares] = [x for x in inf_values if x % 2 != 0]
    result[bit_results].extend(bit_values[:5])

```

```

result[processing_time] = time.time() - start_time
result[cache_size] = BIG_DATA_CACHE.size()

return result

def test_fixed_generators():
    """
    Тестовая функция для демонстрации исправлений
    """
    print("=" * 60)
    print("ЗАПУСК ИСПРАВЛЕННОЙ ВЕРСИИ (Вариант №5)")
    print("=" * 60)

    # Запуск отслеживания памяти
    tracemalloc.start()

    try:
        # Тестовые данные
        test_data = [
            {id: 1, value: 100},
            {id: 2, value: 200},
            {id: 3, value: 300},
            {id: 4, value: 400},
            {id: 5, value: 500}
        ]

        # Вызов исправленной функции
        result = process_data_with_generators_fixed(test_data)

        print("\n=== РЕЗУЛЬТАТ ОБРАБОТКИ ===")
        print(f"Четные квадраты: {result[even_squares]}")
        print(f"Нечетные квадраты: {result[odd_squares]}")
        print(f"Результаты битового сдвига: {result[bit_results]}")
        print(f"Время обработки: {result[processing_time]:.4f} сек")
        print(f"Размер кеша: {result[cache_size]}")

        # Демонстрация корректной обработки StopIteration
        print("\n=== КОРРЕКТНАЯ ОБРАБОТКА STOPITERATION ===")
        gen = (x for x in range(3))
        for i in range(5): # Запрашиваем больше элементов
            try:
                value = next(gen)
                print(f"Элемент {i}: {value}")
            except StopIteration:
                print(f"Элемент {i}: Итерация завершена")
                break

        # Проверка памяти
        snapshot = tracemalloc.take_snapshot()
        print("\n=== СТАТИСТИКА ПАМЯТИ ===")

```

```

print("\nТоп-5 строк по потреблению памяти:")
for i, stat in enumerate(snapshot.statistics(lineno)[:5], 1):
    print(f"{i}. {stat}")

# Демонстрация эффективного использования памяти
print("\n=== ЭФФЕКТИВНОСТЬ ИСПОЛЬЗОВАНИЯ ПАМЯТИ ===")
import gc
gc.collect() # Принудительная сборка мусора
snapshot_after_gc = tracemalloc.take_snapshot()

diff = snapshot_after_gc.compare_to(snapshot, lineno)
print("Изменения в памяти после сборки мусора:")
for stat in diff[:3]:
    print(f" {stat}")

tracemalloc.stop()

except Exception as e:
    print(f"Критическая ошибка: {e}")
    import traceback
    traceback.print_exc()

def demonstrate_generator_best_practices():
    """
    Демонстрация лучших практик использования генераторов
    """
    print("\n" + "=" * 60)
    print("ЛУЧШИЕ ПРАКТИКИ ИСПОЛЬЗОВАНИЯ ГЕНЕРАТОРОВ")
    print("=" * 60)

    # 1. Использование контекстного менеджера для генераторов
    print("\n1. Контекстный менеджер:")
    def read_large_file_chunks(filename: str, chunk_size: int = 1024):
        """Чтение большого файла по частям"""
        try:
            with open(filename, r) as file:
                while True:
                    chunk = file.read(chunk_size)
                    if not chunk:
                        break
                    yield chunk
        except FileNotFoundError:
            print(f"Файл {filename} не найден")
            return

    # 2. Генератор с очисткой ресурсов
    print("\n2. Генератор с очисткой ресурсов:")
    def resource_generator():
        resource = None
        try:

```

```

        resource = "some_resource"
        yield resource
    finally:
        if resource:
            print("Ресурс освобожден")
            resource = None

for item in resource_generator():
    print(f"Использование: {item}")

# 3. Композиция генераторов
print("\n3. Композиция генераторов:")
def square_generator(numbers):
    for n in numbers:
        yield n ** 2

def filter_even_generator(numbers):
    for n in numbers:
        if n % 2 == 0:
            yield n

# Композиция с безопасной обработкой
numbers = range(10)
even_squares = filter_even_generator(square_generator(numbers))
print(f"Четные квадраты: {list(even_squares)}")

# 4. Использование itertools для эффективной обработки
print("\n4. Использование itertools:")
import itertools

# Бесконечный итератор с ограничением
counter = itertools.count(start=1, step=2) # Нечетные числа
limited_odd = itertools.islice(counter, 0, 5) # Берем только 5
print(f"Первые 5 нечетных чисел: {list(limited_odd)}")

# 5. Генератор с кешированием результатов
print("\n5. Генератор с кешированием:")
@lru_cache(maxsize=3)
def cached_computation(n):
    """Вычисление с кешированием"""
    print(f"Вычисление для {n}...")
    return n ** 3

for i in range(5):
    result = cached_computation(i % 3 + 1)
    print(f"cube({i % 3 + 1}) = {result}")

def compare_memory_usage():
    """
    Сравнение использования памяти между старой и новой версией

```

```

"""
print("\n" + "=" * 60)
print("СРАВНЕНИЕ ИСПОЛЬЗОВАНИЯ ПАМЯТИ")
print("=" * 60)

import psutil
import os

process = psutil.Process(os.getpid())
memory_before = process.memory_info().rss / 1024 / 1024 # В МБ

# Тест исправленной версии
print("\nТестирование исправленной версии:")
tracemalloc.start()

# Создаем и обрабатываем большой объем данных
for _ in range(100):
    create_large_data_efficient()
    list(limited_infinite_generator(50))
    list(fixed_bit_generator())

snapshot = tracemalloc.take_snapshot()
top_stats = snapshot.statistics(lineno)[:5]

memory_after = process.memory_info().rss / 1024 / 1024

print(f"Память до: {memory_before:.2f} МБ")
print(f"Память после: {memory_after:.2f} МБ")
print(f"Использовано: {memory_after - memory_before:.2f} МБ")
print("\nТоп-5 строк по памяти:")
for stat in top_stats:
    print(f" {stat}")

tracemalloc.stop()

# Освобождаем память
BIG_DATA_CACHE.clear()
import gc
gc.collect()

memory_final = process.memory_info().rss / 1024 / 1024
print(f"Память после очистки: {memory_final:.2f} МБ")

if __name__ == "__main__":
    # Запуск исправленной тестовой функции
    test_fixed_generators()

    # Демонстрация лучших практик
    demonstrate_generator_best_practices()

```

```
# Сравнение использования памяти
compare_memory_usage()

print("\n" + "=" * 60)
print("ИТОГ ИСПРАВЛЕНИЙ В ВАРИАНТЕ №5")
print("=" * 60)
print("""
    ✓ ИСПРАВЛЕНО:
    1. StopIteration обрабатывается корректно через try-except с break
    2. Сдвиг бит исправлен на << (умножение на 2)
    3. Утечка памяти устранена через:
       - MemoryEfficientCache с ограничением размера
       - Явное удаление больших объектов
       - Использование weakref
       - Сборка мусора
    4. Бесконечный генератор исправлен добавлением условия выхода

    🔧 ДОПОЛНИТЕЛЬНЫЕ УЛУЧШЕНИЯ:
    - Добавлена валидация данных
    - Использование lru_cache для кеширования
    - Композиция генераторов
    - Контекстные менеджеры
    - Эффективное использование памяти

    📊 РЕЗУЛЬТАТЫ:
    - Нет утечек памяти
    - Корректная обработка всех исключений
    - Правильная работа генераторов
    - Эффективное использование ресурсов
    """)
```

Вопросы

A1. Стек вызовов - это структура данных, хранящая информацию о последовательности вызовов функций в момент выполнения программы. Он показывает путь от точки входа до текущей позиции. Увидеть стек вызовов можно: при возникновении ошибки (traceback выводится автоматически); с помощью модуля traceback (traceback.print_stack()); в отладчике pdb командой where или w; с помощью inspect.stack() для программного доступа.

A2. except: перехватывает абсолютно все исключения, включая системные (SystemExit, KeyboardInterrupt, GeneratorExit), что считается антипаттерном, так как может скрыть критические ошибки и помешать корректному завершению программы. except Exception as e: перехватывает только исключения, унаследованные от класса Exception (все стандартные ошибки), но не перехватывает системные исключения, что является правильной практикой.

A3. Использовать команду: python -m pdb script.py. Также можно использовать python -m pdb -c continue script.py для автоматического запуска без остановки в начале, или установить переменную окружения PYTHONBREAKPOINT=0 для отключения breakpoint().

A4. `n` (next) - выполнить следующую строку (не заходя в функции); `s` (step) - войти в функцию; `c` (continue) - продолжить выполнение до следующей точки останова; `p var` - напечатать значение переменной; `pp var` - красивая печать (pretty print) для сложных объектов; `l` (list) - показать код вокруг текущей строки; `up / down` - перемещение по стеку вызовов; `w` (where) - показать текущий стек вызовов; `b` (break) - установить точку останова; `condition N expr` - условная точка останова; `q` (quit) - выйти из отладчика; `h` (help) - справка по командам.

A5. Как установить точку останова (breakpoint) в `pdb` на определенной строке или функции?

В `pdb`: `b 123` - установить на строке 123; `b function_name` - установить на входе в функцию; `b filename:123` - в конкретном файле; `break` - показать все текущие точки останова; `cl 1` - удалить точку останова №1. Также можно использовать `break function_name if condition` для условной точки.

A6. Что такое утечка памяти в Python и как она может возникнуть?

Утечка памяти - это ситуация, когда программа не освобождает память, которая больше не используется, что приводит к росту потребления памяти и возможному падению программы. В Python может возникнуть из-за: замыканий, захватывающих большие объекты; глобальных переменных и кешей без ограничения размера; циклических ссылок (особенно с `__del__`); неосвобожденных ресурсов (файлы, соединения); объектов, хранящихся в списках/словарях без очистки; декораторов с состоянием, сохраняющих данные.

A7. `tracemalloc` - встроенный модуль для отслеживания выделений памяти. Также есть сторонние модули: `memory_profiler` для построчного анализа; `objgraph` для визуализации графов объектов; `guppy3` для анализа кучи; `rumpler` для отслеживания размера объектов.

A8. Использовать `import traceback`; `traceback.print_exc()` для вывода последней ошибки; `traceback.print_exception(type, value, tb)` для полного контроля; `traceback.format_exc()` для получения строки; `logging.exception("Сообщение")` для логирования с `traceback`; в `pdb` команда `where`; также можно использовать `sys.exc_info()` для получения информации об исключении.

A9. `RecursionError` возникает при превышении максимальной глубины рекурсии (по умолчанию 1000 вызовов). Избежать можно: увеличив лимит (`sys.setrecursionlimit(2000)`) - но это опасно; заменив рекурсию на итерацию (циклы); используя хвостовую рекурсию (Python не оптимизирует); добавив базовый случай выхода; используя декораторы для кеширования результатов (`@lru_cache`); для обхода деревьев использовать стек вместо рекурсии.

A10. Команда `up` перемещает текущий контекст на один уровень вверх по стеку вызовов, позволяя просматривать переменные и состояние родительской функции. Используется для: анализа контекста вызова; просмотра локальных переменных на верхних уровнях стека; понимания, откуда была вызвана текущая функция; обратного отслеживания потока выполнения. Противоположная команда - `down` для перемещения вниз.

B1.breakpoint() - встроенная функция Python 3.7+, которая по умолчанию вызывает pdb.set_trace(). Преимущества: может быть переопределена через переменную окружения PYTHONBREAKPOINT (например, PYTHONBREAKPOINT=0 отключает все breakpoint()); может использовать другие отладчики (PYTHONBREAKPOINT=ipdb.set_trace); проще в использовании и короче; стандартный способ для Python 3.7+. pdb.set_trace() - старый способ, работает во всех версиях Python, но менее гибкий.

B2. Использовать условную точку останова: b 123 if var == value - установка точки останова с условием; condition N var == value - добавление условия к существующей точке останова N; tbreak 123 if var == value - временная условная точка (срабатывает один раз). Например, b process_data if counter == 100 остановит выполнение только на 100-й итерации.

B3. Проблемы: засоряет код и его трудно удалять (особенно в больших проектах); требует ручного удаления после отладки; не имеет уровней логирования (DEBUG, INFO, ERROR); не дает информации о времени, модуле, строке вызова; не работает в production окружении (или создает шум); снижает производительность (синхронный вывод); может создавать проблемы с кодировкой; при выводе больших объектов может замедлять программу; сложно отключать для разных модулей. Правильный подход - использование logging.debug() с настройкой уровней.

B4. Стратегия: запустить tracemalloc.start() при старте приложения; снимать снимки памяти через регул

Вывод

В ходе выполнения работы по варианту №5 были выявлены и исправлены четыре типа ошибок: StopIteration без корректной обработки (генератор создавал 3 элемента, а программа запрашивала 5, что приводило к исключению и подстановке мусорных данных), неверный сдвиг бит (использовался оператор >> вместо <<, из-за чего числа делились на 2 вместо умножения на 2), утечка памяти через замыкание (внутренняя функция захватывала большой список из миллиона элементов и сохраняла его в глобальном кеше, не давая сборщику мусора освободить память) и бесконечный генератор (отсутствовало условие выхода из цикла while True, что приводило к зависанию программы). В результате применения методов отладки (pdb, tracemalloc, условные точки останова) все ошибки были успешно локализованы и исправлены, утечки памяти устранены, а программа стала работать корректно с правильной обработкой исключений и эффективным использованием ресурсов.